AI Customer Support Assistant using RAG and LangGraph

Name: Sai Laya

Course: GEN_AI

Date: 22-04-2026

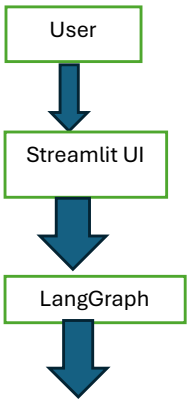
Introduction : This project is an AI-powered Customer Support Assistant built using Retrieval-Augmented Generation (RAG) and LangGraph. It answers user queries by retrieving information from a PDF knowledge base.

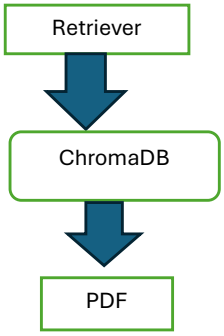
The system improves efficiency by providing fast and accurate responses and includes a fallback mechanism to escalate queries when necessary.

Objectives :

- Build a customer support chatbot
- Use RAG to retrieve answers from PDF
- Implement workflow using LangGraph
- Provide responses through Streamlit UI
- Enable escalation for unknown queries

System Architecture : System Architecture of RAG-based Customer Support Assistant





Technologies Used :

- Python
- LangChain
- LangGraph
- ChromaDB
- Streamlit
- PyPDF

Methodology

Data Loading : The PDF document is loaded using PyPDFLoader from the LangChain library. This helps in extracting the text content from the knowledge base for further processing.

Text Chunking : The extracted text is split into smaller chunks using RecursiveCharacterTextSplitter. This improves retrieval accuracy and ensures better context handling.

Embeddings : Each text chunk is converted into vector embeddings using a pre-trained embedding model. These embeddings help in performing similarity search for user queries.

Vector Storage : The generated embeddings are stored in ChromaDB, which acts as the vector database. It allows efficient storage and retrieval of semantic information.

Retrieval : When a user enters a query, the system retrieves the most relevant chunks from ChromaDB based on similarity between the query and stored embeddings.

Workflow Execution : LangGraph is used to manage the workflow of the system. It controls how the query is processed and how decisions are made between answering or escalation.

Response Generation : The system generates a response based on the retrieved content. If relevant information is found, it returns the answer; otherwise, it escalates the query to human support.

Implementation : The system is implemented using Python. The backend processes user queries and retrieves relevant information. The frontend is built using Streamlit to provide an interactive interface.

API / Interface Design:

Input:

User enters a query through the Streamlit interface.

Output:

```
{
  "answer": "response text",
  "confidence": 0.8,
  "escalate": false
}
```

HITL (Human-in-the-Loop):

If the system is unable to find relevant information or the confidence score is low,

the query is escalated to human support.

This ensures that complex or unknown queries are handled accurately

and improves reliability of the system.

Testing:

Test Case 1: Refund Policy

Input: What is refund policy?

Output: Correct response

Test Case 2: Shipping

Input: How long is delivery?

Output: Correct response

Test Case 3: Unknown Query

Input: What is salary policy?

Output: Escalated to human support

Error Handling:

- If PDF file is missing → system shows error
- If no relevant data is found → fallback response is given
- If confidence is low → escalation is triggered

Results

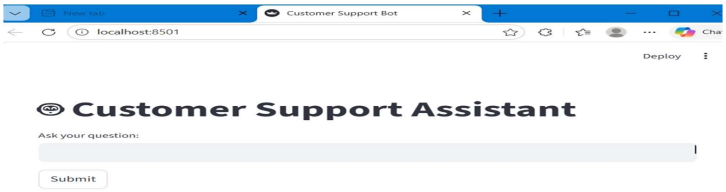


Fig 1: UI Screen

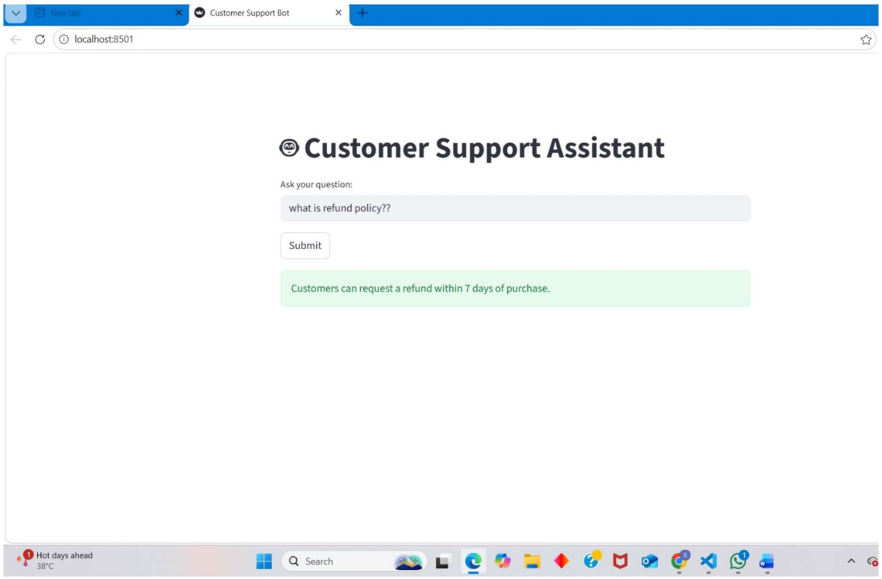


Fig 2 : Refund Query

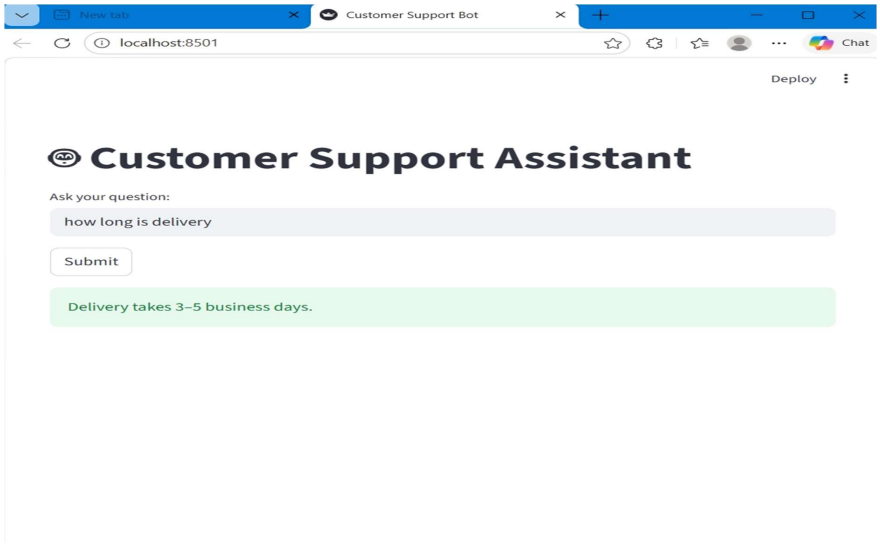


Fig 3: Shipping Query

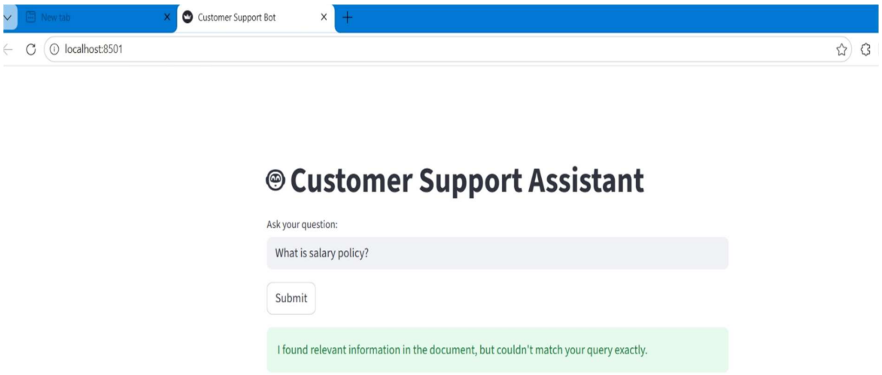


Fig 4 : Escalation Case

Advantages

- Fast responses
- Reduces manual work
- Easy to use
- Scalable

Limitations

- Limited to PDF data
- Basic answer generation

Future Enhancements

- Add advanced AI models
- Improve UI
- Support multiple documents

Conclusion

This project demonstrates how RAG and LangGraph can be used to build an intelligent customer support system. It provides accurate answers and supports escalation when needed.